

UNIVERSIDAD ARGENTINA DE LA EMPRESA

Informe TPO – Entrega 02

Programación III – AJS
Viernes TT

Docente: León, María Ángela

Integrantes

Lee, Agustin LU: 1208427
Ferrario, Juan Ignacio LU: 1211084
Moral Pesce, Sofia Agustina LU: 1208093

Selección de Propuesta y Solución Planteada

La propuesta seleccionada por nuestro grupo fue la PROPUESTA 1 – RED SOCIAL UNIVERSITARIA PARA LA COLABORACIÓN, junto a las funcionalidades de elección obligatorias A1, B1, y B3.

Link al Repositorio: https://github.com/agustnlee/Progra3_2C_2026

La selección de la propuesta fue realizada en base a la familiaridad del equipo con el dominio propuesto, lo cual nos permitirá poder abstraer las consignas apropiadamente y aplicar los algoritmos requeridos de forma práctica.

Nuestra solución consiste en desarrollar un sistema de gestión de una red social universitaria orientada principalmente para su uso desde el punto de vista de un administrador, que podrá gestionar usuarios, publicaciones, conexiones entre usuarios e iniciativas, y ejecutar distintas funcionalidades algorítmicas sobre esos datos. La aplicación será de consola y tendrá como objetivo principal demostrar la aplicación de las estrategias algorítmicas estudiadas en la materia. La información se modelará mediante clases que representan los elementos principales de la red y mediante servicios que contienen la lógica de negocio.

La solución se dividirá en diferentes capas para separar las responsabilidades. Las clases del paquete `model` representarán los datos del sistema, como `Usuario`, `Publicacion`, `Conexion`, `Iniciativa`, `Equipo` y `GrafoUsuarios`. Las clases del paquete `repository` estarán encargadas del almacenamiento de los objetos principales, como usuarios, publicaciones e iniciativas. Las clases del paquete `service` contendrán la lógica de negocio y funcionarán como intermediarias entre la interfaz y los algoritmos. Por último, las clases del paquete `algorithm` contendrán exclusivamente la implementación de los algoritmos seleccionados: `MergeSortPublicaciones`, `DijkstraDifusion`, entre otros . La interfaz estará organizada mediante menús de consola, permitiendo al administrador seleccionar las diferentes operaciones.

El funcionamiento comienza con la carga de los datos de la comunidad universitaria, que pueden ser ingresados manualmente por el administrador o mediante scripts de generación de datos iniciales de prueba. A partir de esos datos se podrán administrar usuarios, publicaciones, iniciativas y conexiones. Cuando el administrador solicita una funcionalidad algorítmica, la interfaz envía la solicitud al servicio correspondiente. El servicio obtiene los datos necesarios desde los repositorios o desde el grafo de usuarios y los entrega al algoritmo correspondiente. El algoritmo procesa la información utilizando la estrategia requerida y devuelve el resultado al servicio, que finalmente lo presenta en la interfaz. De esta forma, existe una separación clara entre los datos, la lógica de negocio, los algoritmos y la interacción con el usuario.

La solución estará orientada principalmente a demostrar el uso correcto de las estrategias algorítmicas. Por eso, los algoritmos están implementados de manera explícita en clases independientes y no se delegará la resolución principal a métodos de bibliotecas que

oculten la estrategia solicitada. Cada funcionalidad podrá ser probada con distintos conjuntos de datos y se analizará su complejidad temporal, tal como requiere la consigna.

Cabe destacar que se utilizaron estructuras de datos y métodos autóctonos de java tales son los casos de List, ArrayList, Map, y sus métodos correspondientes, con el objetivo de simplificar el desarrollo sin la necesidad de implementar TDA específicos (a excepción de GrafoUsuarios).

A continuación se van a realizar las justificaciones de las decisiones relevantes tomadas para el desarrollo de esta primera entrega.

Representación de GrafoUsuarios mediante una lista de adyacencia

El grafo se construye a partir de tres clases. Usuario es el vértice y su identidad depende únicamente de su id (equals y hashCode), por lo que puede usarse como clave de un mapa aunque cambien su nombre o sus habilidades. Conexion es la arista dirigida origen -> destino, que guarda un tiempo en minutos y un indicador de si está habilitada. Al crearse valida que origen y destino existan, que sean distintos y que el tiempo sea positivo, requisito necesario para el funcionamiento de Dijkstra. Una conexión deshabilitada no se elimina: se conserva en el grafo con su información, pero los algoritmos de camino no la utilizan. Por último, GrafoUsuarios es un LinkedHashMap<Usuario, List<Conexion>> que asocia a cada usuario con sus conexiones salientes. Una conexión A -> B se guarda solo en la lista de A y es independiente de B -> A.

Se optó por representar el grafo mediante una lista de adyacencia, en lugar de utilizar una matriz de adyacencia estática. Esta elección se justifica porque la cantidad de usuarios de la red puede crecer y variar durante la ejecución, por lo que no resulta conveniente depender de una estructura de dimensiones fijas que deba dimensionarse según una cantidad máxima de vértices.

Además, al tratarse de un grafo dirigido en el que puede existir como máximo una conexión por cada par ordenado de usuarios, no necesariamente todos los usuarios estarán conectados entre sí. Una matriz de adyacencia requiere (O(V^2)) espacio, incluso cuando existen pocas conexiones, mientras que la lista de adyacencia requiere (O(V+E)), siendo (V) la cantidad de usuarios y (E) la cantidad de conexiones. Por lo tanto, esta representación permite aprovechar mejor la memoria cuando la red es dispersa, es decir, cuando la cantidad de conexiones es considerablemente menor que el máximo posible.

Por último, esta estructura facilita agregar y eliminar usuarios y conexiones, así como recorrer las conexiones salientes de un usuario, operación utilizada por algoritmos como Dijkstra. La implementación permite también mantener conexiones habilitadas y deshabilitadas sin eliminarlas del grafo, conservando la información necesaria para las consultas y los algoritmos.

Funcionalidad Obligatoria 1: Organización del muro de publicaciones

Para este proyecto requerimos de un algoritmo que permita consultar las publicaciones, desde la más reciente hasta la más antigua,desde la más relevante hasta la menos relevante y en caso de empate, se debe priorizar la publicación más reciente y, luego, el identificador menor. Adicionalmente el algoritmo que debemos implementar tiene que ser adecuado para trabajar con volúmenes crecientes de publicaciones, sin utilizar métodos de ordenamiento provistos por Java.

El algoritmo seleccionado para la solución es Merge Sort. El algoritmo presenta una estrategia divide y conquista con una complejidad temporal de O(n log(n)). Al tener estrategia divide y conquista su caso base es una secuencia de un solo elemento, ya que se considera ordenada. El algoritmo divide una secuencia de entrada en dos mitades, y ordena cada una de las mitades por separado para luego mezclar las dos mitades ordenadas en una nueva secuencia ordenada, lo cual dará como resultado la secuencia total ordenada. En nuestro caso estas secuencias de entrada serán las fechas, relevancia y los Id de todas las publicaciones.

Se descartaron los algoritmos de ordenamiento por selección e inserción debido a que presentan una complejidad temporal de O(n^2), lo que resulta menos eficiente para volúmenes crecientes de publicaciones. También se descartó QuickSort porque, si bien ofrece una complejidad promedio de O(n log(n)), su peor caso alcanza O(n^2). En cambio, Merge Sort garantiza una complejidad temporal de O(n log(n)), independientemente del orden inicial de los elementos.

Debido a que al ordenar las publicaciones se utiliza o la fecha, relevancia o Id y siendo que estos datos son respectivamente LocalDateTime e Int no hay diferencia en el algoritmo que se utilice para ordenarlas en este aspecto. Todos los algoritmos de ordenamiento vistos en clase se pueden utilizar y adaptar a estos tipos de dato, por lo tanto nuestra selección es en base a la complejidad temporal.Teniendo en cuenta que el volumen de las publicaciones es creciente, es de suma importancia implementar un algoritmo con la menor complejidad temporal posible y por esta razón decidimos seleccionar MergeSort. otras opciones como ordenamiento por selección, ordenamiento por inserción tienen una complejidad de O(n^2) y por esta razón quedan descartadas ante merge sort. Con respecto a Quicksort, aunque es verdad que en la mayor cantidad de casos tendrá una complejidad temporal de O(n log(n), su complejidad en el peor caso posible es de O(n^2), por ende, la descartamos.

Explicación de la implementación:

La implementación se encuentra en la clase MergeSortPublicaciones, que contiene dos métodos principales:

mergeSort: recibe el arreglo de publicaciones, los índices inicial y final, y el criterio de ordenamiento. Calcula el punto medio y realiza dos llamadas recursivas para ordenar

ambas mitades. El caso base se alcanza cuando ini >= fin, es decir, cuando el segmento contiene cero o un elemento y ya se considera ordenado.

mezclar: recibe el arreglo original, los índices del segmento y el criterio de ordenamiento. Crea un arreglo auxiliar temporal con el tamaño del segmento que se está mezclando y utiliza dos índices para recorrer las mitades previamente ordenadas. En cada iteración selecciona la publicación que debe ocupar la siguiente posición del arreglo auxiliar, según el criterio establecido. Finalmente, copia los elementos del arreglo auxiliar al arreglo original.

La implementación contempla dos criterios de ordenamiento, siendo estos:

Por fecha: ordena las publicaciones desde la más reciente hasta la más antigua. Cuando dos publicaciones tienen la misma fecha y hora, se prioriza aquella con el identificador menor.

Por relevancia: ordena las publicaciones desde la más relevante hasta la menos relevante. Cuando existe un empate, se prioriza la publicación más reciente y, si también coinciden las fechas, aquella con el identificador menor.

Finalmente, la relevancia de cada publicación se calcula mediante la suma de sus reacciones y la cantidad de comentarios, aplicando un peso de 3 a estos últimos (dentro de la clase Publicación).

Relevancia = 1 × reacciones + 3 × cantidad de comentarios

Los comentarios se representan mediante objetos de la clase Comentario, almacenados en un mapa indexado por identificador dentro de Publicacion. Para calcular la relevancia se utiliza la cantidad de comentarios registrados.

Funcionalidad Obligatoria 2: Cadena de difusión más rápida

Consigna e Interpretación

Dentro del proyecto, la funcionalidad de difusión requiere determinar la cadena de menor tiempo entre dos usuarios de la red de colaboración universitaria. Para representar esta situación, se utiliza un grafo dirigido y ponderado, donde cada vértice representa un usuario y cada arista representa una conexión habilitada, cuyo peso indica el tiempo necesario para transmitir información al usuario de destino. El costo total de una cadena corresponde a la suma de los tiempos de sus conexiones. Para resolver esta problemática se seleccionó el algoritmo de Dijkstra, ya que permite encontrar caminos de costo mínimo cuando los pesos son positivos, condición que se cumple al representar tiempos de transmisión. Además, su implementación permite resolver tanto consultas entre un origen y un destino específicos como consultas desde un origen hacia todos los usuarios alcanzables de la red. Se descartaron los algoritmos Prim y Kruskal debido a que no se requiere de un arbol de recubrimiento mínimo entre todos los usuarios.

Funcionamiento y variables relevantes

La implementación se encuentra en la clase DijkstraDifusion y utiliza el grafo representado mediante listas de adyacencia. Al comenzar, se valida que el origen (y el destino, si se indicó) pertenezcan a la red. Luego se inicializan las estructuras distancias, predecesores y fijados, todas vacías, y se registra el origen con distancia cero. En distancias solo figuran los usuarios para los que ya se conoce algún tiempo; un usuario ausente se interpreta como inalcanzado, es decir, con distancia infinita. La estructura predecesores registra el usuario anterior de cada vértice en el camino mínimo encontrado y permite reconstruir la cadena al finalizar, y fijados contiene los usuarios cuya distancia ya es definitiva.

Para procesar los usuarios en orden de distancia acumulada, se emplea una cola de prioridad que ordena las entradas por menor tiempo conocido y, ante empates, por identificador de usuario, lo que hace el resultado determinista. En cada iteración se extrae la entrada de menor tiempo. Si su usuario ya estaba fijado, se trata de una entrada desactualizada (ya se había encontrado una distancia mejor) y se descarta. En caso contrario, el usuario se fija con ese tiempo, que es su mínimo definitivo, y se recorren sus conexiones habilitadas. Para cada conexión cuyo destino aún no esté fijado, se calcula una distancia tentativa sumando el tiempo acumulado del usuario actual con el peso de la conexión. Si es menor que la almacenada (o si el destino no tenía distancia), se actualizan distancias y predecesores y se incorpora una nueva entrada a la cola; si no, la conexión se descarta. La implementación también utiliza una lista de pasos para registrar las decisiones tomadas y facilitar el seguimiento de la ejecución.

Si se consulta un destino específico, el algoritmo finaliza apenas ese destino se fija, ya que en ese momento su distancia es la mínima definitiva. En ese caso se eliminan de distancias los usuarios no fijados, que solo tenían valores provisorios, para no informar tiempos que no son mínimos. En cambio, si se busca desde un origen hacia todos los usuarios, continúa procesando hasta agotar los vértices alcanzables. Finalmente, el objeto Resultado permite

consultar si un destino es alcanzable, obtener su tiempo mínimo y reconstruir el camino a partir de predecesores.

Complejidad Temporal y Espacial

Para analizar la complejidad, se considera V como la cantidad de usuarios o vértices del grafo y E como la cantidad de conexiones o aristas. En el peor caso, que ocurre cuando se procesan todos los usuarios alcanzables desde el origen, cada vértice se fija una única vez y cada conexión habilitada se examina una única vez, al momento de fijar su vértice de origen. Cada vez que se encuentra una distancia mejor se inserta una nueva entrada en la cola de prioridad, operación de costo logarítmico, y como a lo sumo se insertan E entradas (más la del origen), también se realizan a lo sumo E + 1 extracciones, de las cuales las entradas desactualizadas se descartan en tiempo constante. La complejidad temporal en el peor caso es O((V + E) log V), que corresponde a la implementación de DijkstraDifusion con listas de adyacencia y cola de prioridad. La construcción de la lista de conexiones habilitadas de cada vértice cuesta O(grado) y, sumada sobre todos los vértices fijados, aporta O(E), por lo que no modifica la cota.

En el mejor caso, cuando se consulta un destino que coincide con el origen, el algoritmo comienza con estructuras vacías en las que solo se registra el origen, lo extrae de la cola, lo fija y finaliza de forma anticipada sin recorrer ninguna conexión. Por lo tanto, el costo temporal es O(1). No se asigna una complejidad única al caso promedio, ya que depende de la estructura del grafo, de la distribución de las conexiones y de los destinos consultados; en grafos ralos, donde E es del orden de V, el comportamiento se aproxima a O(V log V).

En cuanto a la complejidad espacial, las estructuras de distancias, predecesores y vértices fijados requieren espacio proporcional a V, mientras que la cola de prioridad puede acumular hasta O(E) entradas por la presencia de entradas desactualizadas. El registro de pasos crece con un elemento por cada vértice fijado y uno por cada conexión examinada, es decir, O(V + E). Por ello, la complejidad espacial auxiliar es O(V + E). En conclusión, DijkstraDifusion permite resolver consultas de caminos mínimos de difusión con un costo eficiente, y su ejecución puede finalizar anticipadamente cuando se busca un destino específico.

Funcionalidad Obligatoria 3: Selección de iniciativas para financiar

Consigna e Interpretación

La tercera propuesta nos permite seleccionar un conjunto de iniciativas para financiar, buscando maximizar el impacto total sin superar un presupuesto disponible. Cada una de las iniciativas contiene un identificador, un nombre, un costo entero (no fraccionario) y un valor de impacto. A partir de una lista de estas iniciativas, más un presupuesto máximo, el sistema determina qué alternativas deben seleccionarse y devolviendo el resultado de la financiación, incluyendo el cuales fueron seleccionadas, el costo total utilizado, el impacto total y el presupuesto restante. Para la solución de esta propuesta se implementó el algoritmo de la Mochila 0/1 mediante programación dinámica.

Estrategia

Se eligió Programación Dinámica porque el problema puede dividirse en subproblemas que consideran una cantidad creciente de iniciativas y diferentes límites de presupuesto. Los resultados parciales se almacenan en una matriz para evitar resolver repetidamente los mismos subproblemas. La estrategia permite obtener una solución óptima sin enumerar explícitamente todas las combinaciones posibles de iniciativas.

Se descartó una estrategia Greedy porque seleccionar iniciativas según su impacto individual, su relación impacto/costo u otro criterio local no garantiza obtener la combinación de mayor impacto total.

Aplicación

La implementación se encuentra en la clase MochilaDinamica, donde se llevó a cabo utilizando una matriz dp de dimensiones ((n+1)(B+1)), donde “n” representa la cantidad de iniciativas y “B” el presupuesto disponible.
Cada posición dp[i][p] almacena el máximo impacto que puede obtenerse considerando las primeras (i) iniciativas y un presupuesto máximo de (p).
Para cada iniciativa, el algoritmo evalúa dos alternativas: no seleccionarla, conservando el impacto máximo anterior, o si la selecciona, suma su impacto al mejor resultado alcanzable con el presupuesto restante. (La comparación entre alternativas se realiza mediante Math.max()).
Luego de completar la matriz, se reconstruye la solución recorriendo sus filas en sentido inverso, identificando las iniciativas que forman parte de la combinación óptima.
Por último, MochilaDinamicaService calculará el costo total, el impacto obtenido y el presupuesto restante, devolviendo estos valores mediante un objeto ResultadoFinanciación.

Complejidad

Sean (n) la cantidad de iniciativas y (B) el presupuesto disponible.

La matriz contiene ((n+1)(B+1)) estados y cada uno se calcula mediante una cantidad constante de operaciones. Por lo tanto, la complejidad temporal es: (O(n.B))
La matriz requiere (O(n.B)) espacio adicional. La reconstrucción de la solución y el cálculo de los totales requieren, como máximo, (O(n)), por ende no modifican la complejidad asintótica. La complejidad es pseudo polinómica, ya que depende del valor numérico del presupuesto, el algoritmo resulta adecuado para cantidades moderadas de iniciativas y presupuestos enteros.

Funcionalidad Optativa A1: Consultas de difusión entre todos los usuarios

Consigna e Interpretación

Dentro del proyecto, la funcionalidad de difusión requiere determinar los tiempos mínimos de transmisión entre todos los pares de usuarios de la red de colaboración universitaria. Para representar esta situación, se utiliza un grafo dirigido y ponderado, donde cada vértice representa un usuario y cada arista representa una conexión habilitada, cuyo peso indica el tiempo necesario para transmitir información al usuario de destino. El costo total de una cadena corresponde a la suma de los tiempos de sus conexiones. Para resolver esta problemática se seleccionó el algoritmo de Floyd, ya que permite calcular los caminos de costo mínimo entre todos los pares de vértices cuando los pesos de las aristas son no negativos. A diferencia de Dijkstra, que resuelve consultas desde un origen, Floyd permite obtener los tiempos mínimos para toda la red en una única ejecución, lo que resulta adecuado para realizar consultas repetidas entre distintos pares de usuarios.

Funcionamiento y variables relevantes

La implementación se encuentra en la clase FloydDifusion y utiliza el grafo representado mediante listas de adyacencia como fuente de información para inicializar las matrices de cálculo. Al comenzar, se construye la matriz distancias, que almacena el menor tiempo conocido entre cada par de usuarios. Se asigna cero a las posiciones de la diagonal principal, que representan el tiempo de un usuario hacia sí mismo; infinito a los pares que inicialmente no tienen una conexión directa; y el tiempo correspondiente a las conexiones habilitadas para los pares que sí están conectados. También se utiliza la matriz siguiente, que registra el próximo usuario que debe recorrerse desde un origen para reconstruir el camino mínimo hacia un destino.

El algoritmo evalúa progresivamente cada vértice como posible intermediario de los caminos. Para ello, utiliza las variables k, i y j, que representan, respectivamente, el vértice intermediario, el usuario de origen y el usuario de destino. En cada iteración se compara el tiempo conocido entre i y j con el costo de llegar desde i hasta k y continuar desde k hasta j. Si este último resulta menor, se actualiza distancias[i][j] y se modifica siguiente[i][j] para conservar la información necesaria para reconstruir el camino encontrado. De esta manera, la matriz incorpora progresivamente caminos que utilizan distintos vértices como intermediarios, hasta determinar los tiempos mínimos entre todos los pares.

Al finalizar, los resultados permiten consultar el tiempo mínimo de difusión entre dos usuarios y reconstruir la cadena correspondiente mediante la matriz siguiente. Si no existe un camino entre el origen y el destino, el resultado indica que son inalcanzables. Además, la clase FloydService coordina las consultas y puede reutilizar los resultados calculados para evitar ejecutar nuevamente el algoritmo ante cada solicitud. Para mantener la consistencia, debe detectar las modificaciones del grafo e invalidar la caché cuando cambia su versión.

Complejidad Temporal y Espacial

Para analizar la complejidad, se considera V como la cantidad de usuarios o vértices del grafo y E como la cantidad de conexiones o aristas. En el peor caso, la complejidad temporal de Floyd es O(V ^ 3), ya que el algoritmo utiliza tres ciclos anidados que recorren los vértices como intermediarios, orígenes y destinos. Para cada combinación, evalúa si utilizar un vértice intermediario permite mejorar el tiempo mínimo conocido entre dos usuarios. Por lo tanto, el número de operaciones de actualización de distancias es proporcional a V ^ 3.

En el mejor caso, la complejidad temporal es O(V ^ 2), ya que sin conexiones, la poda evita el ciclo interno. Aunque las matrices de resultados se inicializan con un costo de O(V ^ 2), esta etapa queda dominada por el procesamiento de los tres ciclos. En consecuencia, tanto el mejor como el peor caso tienen un orden de crecimiento cúbico. Para el caso promedio, tampoco corresponde establecer una cota asintótica más ajustada sin especificar supuestos particulares sobre la implementación y los datos de entrada; bajo la implementación estándar, el costo continúa siendo O(V ^ 3).

En cuanto a la complejidad espacial, las matrices distancias y siguientes almacenan información para cada par de usuarios. Cada una requiere un espacio de O(V ^ 2), por lo que la complejidad espacial auxiliar total es O(V ^ 2), sin considerar el espacio ocupado previamente por el grafo. En conclusión, Floyd permite calcular los tiempos mínimos de difusión entre todos los pares de usuarios, con un costo temporal cúbico y un costo espacial cuadrático. Su principal ventaja es que, una vez calculados los resultados, las consultas entre distintos pares pueden resolverse eficientemente sin repetir el algoritmo, siempre que los resultados almacenados continúen siendo válidos.

Funcionalidad Optativa B1: Formación de equipos con restricciones

Consigna e Interpretación

Para el algoritmo B1 requerimos de un algoritmo que permitiese la formación de equipos de estudiantes distintos donde se pudiese restringir la cantidad de estudiantes por equipo, las habilidades de los estudiantes en dicho equipo, la inclusión de estudiantes obligatorios y la exclusión de pares de estudiantes particulares. El algoritmo debía encontrar y mostrar todos los equipos válidos, sin repetir el mismo equipo por presentar sus integrantes en otro orden.

Teniendo en cuenta estas condiciones escogimos un algoritmo de tipo backtracking, porque el problema requiere encontrar todas las combinaciones posibles de estudiantes que satisfagan las restricciones establecidas. Esta estrategia permite construir progresivamente cada equipo, evaluando las decisiones tomadas y descartando aquellas alternativas que incumplan las condiciones. Cuando se completa un equipo válido, se almacena y la búsqueda continúa para encontrar las demás soluciones.

Se descartó Greedy porque la elección local de estudiantes no garantiza encontrar todos los equipos válidos. Tampoco se seleccionó Programación Dinámica, ya que el problema requiere enumerar todas las combinaciones que cumplan las restricciones, no optimizar una solución mediante subproblemas.

Implementación

La implementación se encuentra en la clase BacktrackingSubconjuntosEquipo, que contiene los métodos ejecutar, Backtracking y EquipoValido.

El método ejecutar recibe la lista de candidatos y un objeto RestriccionesEquipo, que encapsula el tamaño requerido del equipo, las habilidades necesarias, los integrantes obligatorios y los pares incompatibles. A partir de estos datos prepara las estructuras auxiliares e inicia la búsqueda mediante el método Backtracking.

El método Backtracking recibe diez parámetros: un arreglo de índices que representa el equipo en construcción (usuarios), el nivel actual de recursión (E), la cantidad de candidatos (n), el arreglo de usuarios disponibles (todos), el tamaño requerido del equipo (k), las habilidades requeridas (habilidadesReq), los integrantes obligatorios (obligatorios), los pares incompatibles (incompatibles), la lista de equipos encontrados (equipos) y el registro de pasos de ejecución (pasos).

En cada nivel, el algoritmo recorre los candidatos disponibles y utiliza el arreglo usuarios para registrar sus índices. Para evitar repetir integrantes y generar el mismo equipo en distintos órdenes, se exige que los índices seleccionados sean estrictamente crecientes. Cuando un candidato permite continuar la construcción de un equipo válido hasta ese nivel, se realiza una nueva llamada recursiva.

El método EquipoValido verifica las restricciones durante la construcción. Primero comprueba que no existan integrantes repetidos y que no se incluya ningún par incompatible. Una vez completado el equipo, también verifica que estén presentes todos los integrantes obligatorios y que el conjunto de habilidades aportadas por sus integrantes cubra la totalidad de las habilidades requeridas. Es decir, no es necesario que cada estudiante posea individualmente todas las habilidades, sino que estas estén cubiertas por el equipo en su conjunto.

Cuando se alcanza el tamaño requerido y se cumplen todas las condiciones, se construye un objeto Equipo y se incorpora a la lista de resultados. El algoritmo continúa explorando las alternativas restantes hasta finalizar la búsqueda. Adicionalmente, se registra información sobre la ejecución mediante la lista pasos, limitada a un máximo de 200 registros.

Finalmente, el objeto Resultado permite obtener los equipos válidos encontrados y los pasos registrados durante la ejecución.

Complejidad Temporal y Espacial

Siendo n la cantidad total de candidatos, k el tamaño del equipo, I la cantidad de pares incompatibles, U la cantidad de usuarios obligatorios, h la cota superior de habilidades que posee cada usuario individual, y H la cantidad de habilidades requridas:

La complejidad temporal es de O(k * C(n, k) * (k + I) + C(n, k) * (U + k * h + H)). El algoritmo explora combinaciones sin repetición mediante índices crecientes, generando a lo sumo O(C (n, k)) hojas/estados terminales en el árbol de recursión. Los nodos intermedios cuestan O(k + I) (orden + incompatibles). Las hojas terminales agregan O(U + k * h + H)

En términos asintóticos simplificados, el peor caso ocurre cuando k ≈ n/2, donde el número de combinaciones alcanza su máximo y la complejidad resulta de orden exponencial, O(2^n). En contraste, si k es significativamente menor que n (o se considera constante), la combinatoria está acotada por C(n, k) ≤ (n^k)/k!, lo que reduce la complejidad a un comportamiento polinomial de orden O(n ^ k)

La complejidad espacial auxiliar es de O(k*h), determinada por la profundidad máxima de la pila de llamadas (k) y las estructuras locales temporales utilizadas en cada nivel (conjuntos de hasta k integrantes y hasta k * h habilidades acumuladas), con un consumo constante O(1) para el registro de pasos (acotado a 200). Sin embargo, considerando la salida requerida, el algoritmo almacena todos los resultados válidos en la lista equipos, lo que demanda hasta O(k * C(n, k)) de memoria para albergar hasta O(C(n,k)) combinaciones de k integrantes en el peor caso.

Funcionalidad Optativa B3: Cadenas alternativas de difusión

Análisis del problema

El problema consiste en encontrar todas las cadenas de difusión posibles entre un usuario de origen y uno de destino, respetando determinadas restricciones establecidas por el sistema. La red de difusión se representa mediante un grafo dirigido y ponderado, donde cada usuario constituye un vértice y cada conexión de difusión representa una arista cuyo peso indica el tiempo necesario para transmitir la información entre dos usuarios. El objetivo es obtener todas las cadenas que cumplan simultáneamente estas condiciones, calcular el tiempo total de cada una e identificar aquella que presente el menor tiempo de difusión.

Para resolver el problema se reciben los siguientes datos:

- **Origen:** usuario desde el cual comienza la difusión.
- **Destino:** usuario al que se desea hacer llegar la información.
- **Tiempo máximo:** límite de tiempo total permitido para una cadena de difusión.
- **Máximo de intermediarios:** cantidad máxima de usuarios que pueden participar entre el origen y el destino.
- **Usuarios excluidos:** conjunto de usuarios que no pueden formar parte de ninguna cadena válida.

Las restricciones que deben cumplirse son:

1. No se puede repetir un mismo usuario dentro de una cadena.
2. No se pueden atravesar usuarios incluidos en el conjunto de excluidos.
3. La cantidad de intermediarios no puede superar el límite establecido.
4. El tiempo total de las conexiones utilizadas no puede superar el tiempo máximo permitido.
5. Solamente se pueden utilizar conexiones habilitadas en el grafo.

Estrategia: Backtracking mediante recorrido en profundidad (DFS).

Se seleccionó esta técnica porque el problema requiere encontrar todas las posibles cadenas de difusión que cumplan las determinadas restricciones, en lugar de limitarse a encontrar un único camino entre el origen y el destino, además, permite construir progresivamente una solución, tomando decisiones sobre qué usuario visitar a continuación, y cuando una alternativa no cumpla alguna de las restricciones, se descarta sin continuar explorando. La búsqueda se realiza mediante un recorrido en profundidad (DFS), avanzando recursivamente por las conexiones disponibles desde cada usuario. Cuando se alcanza el destino, se registra la cadena encontrada y se retrocede para continuar explorando otras alternativas.

La elección resulta la más adecuada porque nos permite explorar todas las combinaciones válidas, descartar anticipadamente aquellas que incumplan las condiciones y conservar las soluciones encontradas para compararlas posteriormente. A diferencia de una estrategia

Greedy, no se toma únicamente la conexión que parece más conveniente en cada paso, ya que hacer esto, podría dejar fuera cadenas que aunque comiencen con conexiones de mayor tiempo, terminan teniendo un tiempo total menor.

Implementación del algoritmo

La implementación se encuentra en la clase BacktrackingCadenasDifusion, dentro del paquete algorithm.

Se reutiliza la estructura de datos GrafoUsuarios, que representa la red mediante listas de adyacencia. Cada usuario mantiene sus conexiones salientes y cada conexión contiene el usuario de destino, el tiempo de difusión y su estado de habilitación.

Para realizar la búsqueda se utilizan las siguientes estructuras auxiliares:

- **caminoActual:** almacena los usuarios que forman parte de la cadena que se está construyendo.
- **visitados:** conjunto que permite controlar que un usuario no se repita dentro de la cadena actual.
- **tiempoAcumulado:** mantiene la suma de los tiempos de las conexiones recorridas.
- **cadenas:** almacena todas las cadenas válidas encontradas.
- **menorTiempo:** conserva la cadena que presenta el menor tiempo total entre las soluciones encontradas.

Funcionamiento

El algoritmo comienza incorporando el usuario de origen a caminoActual y visitados, iniciando el recorrido desde ese punto. Luego, examina las conexiones habilitadas del usuario actual y evalúa cada posible alternativa.

Primero se verifica que el usuario de destino de la conexión no esté excluido ni haya sido visitado anteriormente. Luego, se comprueba que agregarlo no supere la cantidad máxima de intermediarios, salvo cuando se trata del destino. Y por último, se verifica que el tiempo acumulado, sumado al tiempo de la nueva conexión, no supere el máximo permitido.

Cuando una alternativa cumple todas las restricciones, se agrega el siguiente usuario a caminoActual y visitados, se actualiza el tiempo acumulado y se realiza una nueva llamada recursiva desde ese usuario.

Caso base: cuando el usuario actual coincide con el destino, se considera que se encontró una cadena válida, para esto, se guarda una copia de los usuarios recorridos junto con su tiempo total y se compara su duración con la mejor solución encontrada hasta ese momento. Una vez finalizada la exploración de una alternativa, se realiza el retroceso: se elimina el último usuario agregado tanto de caminoActual como de visitados. De esta manera, se recupera el estado anterior, y se puede continuar explorando otra conexión sin arrastrar decisiones de la alternativa previa. Este procedimiento se repite hasta explorar todas las alternativas posibles que respeten las restricciones.

Obtención de la cadena de menor tiempo

Durante la búsqueda, cada vez que se encuentra una cadena válida se compara su tiempo total con el de la mejor solución registrada. Si todavía no existe una solución o el nuevo tiempo es menor, se actualiza menorTiempo. Una vez finalizado el recorrido, se dispone de todas las cadenas válidas y de la que presenta el menor tiempo total.

Complejidad temporal y espacial

Sea V la cantidad de vértices (usuarios) del grafo y E la cantidad de aristas (conexiones).

Complejidad temporal: O(V!).

En el peor caso, el algoritmo debe explorar todas las cadenas simples posibles entre el origen y el destino. Como no se permite repetir usuarios, la cantidad de alternativas crece factorialmente a medida que aumenta la cantidad de usuarios disponibles.

Los controles de tiempo máximo, intermediarios y usuarios excluidos permiten podar ramas y reducir la cantidad de alternativas exploradas en la práctica. Sin embargo, en el peor escenario, cuando las restricciones permiten recorrer prácticamente todas las cadenas simples, la complejidad temporal es factorial.

De manera más precisa, para un límite de k intermediarios, la profundidad máxima de búsqueda es de k + 1 conexiones. Por lo tanto, un límite pequeño de intermediarios puede reducir considerablemente el espacio de búsqueda.

Complejidad espacial auxiliar: O(V).

Se debe considerar el espacio utilizado por caminoActual, visitados y la pila de llamadas recursivas. Como no se repiten usuarios, la profundidad máxima de la recursividad está limitada por la cantidad de vértices. Además, el almacenamiento de las soluciones encontradas requiere O(R · V), donde R representa la cantidad de cadenas válidas obtenidas, ya que cada una puede contener hasta V usuarios.

Pruebas

Las pruebas del proyecto se organizan dentro del directorio src/test/java, separado del código principal ubicado en src/main/java. Esta separación permite verificar el comportamiento de los algoritmos y servicios sin depender exclusivamente de la interacción con la interfaz de consola.

Dentro del directorio de pruebas se encuentra TestRunner.java, utilizado como punto de ejecución de las pruebas desarrolladas. A través de este componente se organizaron las llamadas a las verificaciones de las funcionalidades implementadas y contrastar los resultados obtenidos con los esperados.

Las pruebas permitirán verificar tanto los resultados obtenidos como el cumplimiento de las restricciones de cada problema. También contribuirán a detectar errores de integración y a comprobar que las distintas funcionalidades trabajan correctamente sobre el modelo de datos compartido.

Interfaz de Usuario

Se optó por utilizar un panel de gestión por CLI dado el foco del trabajo práctico sobre los algoritmos. Se decidió implementarlo con una estructura de módulos, en donde cada entidad del dominio (como Usuario, Equipos, etc) tiene su propio módulo. En la carpeta de UI se encuentra Consola.java que contiene métodos reutilizables para la muestra, registro de entradas, y listado de datos. Además se encuentra MenuPrincipal.java que contiene todos los módulos CLI, y muestra la pantalla inicial que le permite al usuario navegar por los diferentes módulos, los cuales tienen sus propias opciones para ejecutar. Cabe destacar que en este caso, los módulos de UI se encargan de la coordinación de los servicios y la muestra de sus resultados.